

PHP CRUD Practice File

[register.php](#) · [login.php](#) · [index.php](#) · [edit.php](#) · [update.php](#)

[Password Encryption](#) · [Error Handling](#) · [Session Auth](#) · [Full CRUD](#)



Overview & How This Guide Works

This file shows you how to build a complete CRUD system in PHP step by step. Every single line of code has a comment explaining what it does and why.

What is CRUD?

Letter	Meaning in This Project
C — Create	register.php → Add a new user to the database
R — Read	index.php → Show all users from the database
U — Update	edit.php + update.php → Change an existing user's data
D — Delete	index.php → Remove a user from the database

Files in This Project

File	Purpose
db.php	Database connection — included in every other file
register.php	Form to add a new user + insert into database
login.php	Form to log in + session creation
index.php	Show all users in a table + delete button
edit.php	Form pre-filled with user data for editing
update.php	Processes the edit form + updates database
logout.php	Destroys session + redirects to login



Password Rule

Passwords are NEVER stored as plain text.
We use PHP's `password_hash()` function to encrypt them.
To check a password we use `password_verify()` — not `==`



Database Setup

Step 1 — Create the Database (phpMyAdmin SQL Tab)

Open phpMyAdmin → Click SQL tab → Paste and run:

```
-- =====
-- DATABASE SETUP
-- Run this ONE TIME in phpMyAdmin SQL tab
-- =====

-- 1. Create the database
CREATE DATABASE IF NOT EXISTS crud_app;

-- 2. Select (use) the database
USE crud_app;

-- 3. Create the users table
CREATE TABLE users (

    -- id: auto-incremented unique number for each row
    id          INT          AUTO_INCREMENT PRIMARY KEY,

    -- name: full name of the user, max 100 characters, cannot be empty
    name        VARCHAR(100) NOT NULL,

    -- email: must be unique – no two users can have the same email
    email        VARCHAR(100) NOT NULL UNIQUE,

    -- password: stores the HASHED password (never plain text)
    -- 255 chars because password_hash() output is about 60 chars but
    -- it is good practice to allow more space
    password     VARCHAR(255) NOT NULL,

    -- phone: optional, so we allow NULL
    phone        VARCHAR(20)  NULL,

    -- role: either 'admin' or 'user', default is 'user'
    role         ENUM('admin','user') DEFAULT 'user',

    -- created_at: automatically saves the date/time of registration
    created_at   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP

);

-- 4. Check the table was created correctly
DESCRIBE users;
```

Table Structure at a Glance

Column	Type	Constraint	Description
id	INT	AUTO_INCREMENT PK	Unique row number – set automatically
name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	NOT NULL UNIQUE	Login email – must be unique
password	VARCHAR(255)	NOT NULL	Hashed password (never

			plain text)
phone	VARCHAR(20)	NULL	Optional phone number
role	ENUM	DEFAULT 'user'	Either 'admin' or 'user'
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on registration

⚠ Why ENUM for role?

ENUM limits values to a fixed list. Role can only be 'admin' or 'user'. MySQL rejects any other value automatically.



db.php — Database Connection



Purpose

This file creates the \$conn variable that connects PHP to MySQL. Every other file includes this file so they all share the same connection.



Full Code with Comments

```

<?php
// =====
// db.php – DATABASE CONNECTION FILE
// Include this file in every page that needs the database:
// require_once 'db.php';
// =====

// -- Configuration constants --
// Using define() so these values cannot be changed later
define('DB_HOST', 'localhost'); // Server where MySQL runs
define('DB_USER', 'root');      // MySQL username (default in XAMPP)
define('DB_PASS', '');          // MySQL password (empty in XAMPP)
define('DB_NAME', 'crud_app');  // The database we created above

// -- Create connection using OOP style MySQLi --
// new mysqli(host, user, password, database)
$conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);

// -- Error handling: stop script if connection fails --
// connect_error is NULL if connection is OK
// If it is NOT null it means there is a problem
if ($conn->connect_error) {
    // die() stops the script and prints the error message
    die('Connection Failed: ' . $conn->connect_error);
}

// -- Set character encoding to UTF-8 --
// This makes sure special characters (é, ñ, etc.) work correctly
$conn->set_charset('utf8mb4');

// -- Helper function: sanitize user input --
// Always call this before using any user input in HTML
function clean($data) {
    // trim()          removes spaces from both ends
    // stripslashes() removes backslashes
    // htmlspecialchars() converts < > & to safe HTML entities
    //                this prevents XSS (cross-site scripting) attacks
    return htmlspecialchars(stripslashes(trim($data)));
}

// -- Helper function: redirect to another page --
function redirect($url) {
    // header() sends a browser instruction to go to $url
    // exit stops any code after the redirect
    header('Location: ' . $url);
    exit;
}
?>

```



What \$conn gives you

Method / Property	What it Does
-------------------	--------------

<code>\$conn->query(sql)</code>	Run a SELECT query, returns result
<code>\$conn->prepare(sql)</code>	Prepare a safe statement (no SQL injection)
<code>\$conn->real_escape_string(x)</code>	Escape a string for use in SQL
<code>\$conn->error</code>	Last error message from MySQL
<code>\$conn->insert_id</code>	ID of the last inserted row
<code>\$conn->affected_rows</code>	Number of rows changed by last query



Common Error

'Access denied for user root' → Wrong username or password in DB_USER / DB_PASS

'Unknown database crud_app' → Run the CREATE DATABASE SQL first in phpMyAdmin



register.php — Add New User (CREATE)



Purpose

Displays a registration form.

On submit: validates input, checks for duplicate email, hashes password, inserts into DB.

This is the C in CRUD (Create).



Password Encryption

`password_hash($password, PASSWORD_DEFAULT)`

→ Creates a secure bcrypt hash like: `$2y$10$abc123...`

→ The hash is different every time even for the same password.

→ NEVER store plain text passwords.



Full Code with Comments

```
<?php
// =====
// register.php – USER REGISTRATION (CREATE)
// =====

// Start session at the VERY TOP before any HTML or echo
session_start();

// Include the database connection file
// require_once: stops if file not found (safer than include)
require_once 'db.php';

// -- Initialize variables --
// We set these empty so the form fields do not show PHP errors
// when the page first loads (before any form submit)
$errors = []; // array to collect validation error messages
$success = ''; // success message string
$name = ''; // will hold the name field value
$email = ''; // will hold the email field value
$phone = ''; // will hold the phone field value

// -- Only run this block when the form is submitted --
// $_SERVER['REQUEST_METHOD'] is 'POST' when form is submitted
// It is 'GET' when the page is first opened
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // -- Read and clean form values --
    // clean() is our helper function from db.php
    // It trims spaces and prevents XSS attacks
    $name = clean($_POST['name']);
    $email = clean($_POST['email']);
    $phone = clean($_POST['phone']);
    $password = $_POST['password']; // Do NOT clean passwords
    $confirm = $_POST['confirm_password']; // We need the raw value

    // — VALIDATION —

    // Check 1: name must not be empty
    if (empty($name)) {
        $errors[] = 'Full name is required.';
    }

    // Check 2: email must not be empty AND must be valid format
    // filter_var() with FILTER_VALIDATE_EMAIL checks the format
    if (empty($email)) {
        $errors[] = 'Email address is required.';
    }
}
```

```

} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Email address format is not valid.';
}

// Check 3: password must be at least 8 characters
if (strlen($password) < 8) {
    $errors[] = 'Password must be at least 8 characters.';
}

// Check 4: both password fields must match
if ($password !== $confirm) {
    $errors[] = 'Passwords do not match.';
}

// — DATABASE CHECK —————

// Only check the database if all form checks passed
// (No point querying DB if basic validation already failed)
if (empty($errors)) {

    // Check 5: email must not already exist in the database
    // We use a prepared statement to prevent SQL injection
    $stmt = $conn->prepare(
        'SELECT id FROM users WHERE email = ?'
    );
    // 's' means the parameter is a String
    $stmt->bind_param('s', $email);
    $stmt->execute();
    // store_result() lets us use num_rows
    $stmt->store_result();

    if ($stmt->num_rows > 0) {
        // Email already registered
        $errors[] = 'This email is already registered.';
    }
    $stmt->close(); // Always close prepared statements
}

// — INSERT INTO DATABASE —————

// Only insert if there are ZERO errors
if (empty($errors)) {

    // HASH the password before storing
    // PASSWORD_DEFAULT uses bcrypt – the most secure option
    // The hash looks like: $2y$10$... (about 60 characters)
    $hashed = password_hash($password, PASSWORD_DEFAULT);

    // Prepare the INSERT statement
    // ? placeholders prevent SQL injection
    $stmt = $conn->prepare(
        'INSERT INTO users (name, email, password, phone) VALUES (?, ?, ?, ?)'
    );
    // 'ssss' = four String parameters (name, email, password, phone)
    $stmt->bind_param('ssss', $name, $email, $hashed, $phone);

    if ($stmt->execute()) {
        // Insert was successful
        $success = 'Registration successful! <a href="login.php">Login here</a>.';
        // Clear the form fields after success
        $name = $email = $phone = '';
    } else {
        // MySQL reported an error
        $errors[] = 'Database error: ' . $conn->error;
    }
    $stmt->close();
}
}

```

```

}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Register</title>
</head>
<body>

<h2>Create Account</h2>

<!-- Show SUCCESS message (green) -->
<?php if ($success): ?>
    <p style="color:green"><?php echo $success; ?></p>
<?php endif; ?>

<!-- Show ERROR messages (red) – one per line -->
<?php if (!empty($errors)): ?>
    <div style="color:red">
        <?php foreach ($errors as $err): ?>
            <p>✖ <?php echo $err; ?></p>
        <?php endforeach; ?>
    </div>
<?php endif; ?>

<!-- FORM: method=post sends data hidden in the request body -->
<!-- action='' means submit to the same file (register.php) -->
<form method="post" action="">

    <!-- value=... re-fills the field if there was an error -->
    Full Name: <br>
    <input type="text" name="name"
        value="<?php echo $name; ?>" required><br><br>

    Email Address: <br>
    <input type="email" name="email"
        value="<?php echo $email; ?>" required><br><br>

    Phone (optional): <br>
    <input type="text" name="phone"
        value="<?php echo $phone; ?>"><br><br>

    Password (min 8 chars): <br>
    <input type="password" name="password" required><br><br>

    Confirm Password: <br>
    <input type="password" name="confirm_password" required><br><br>

    <input type="submit" value="Register">
    <a href="login.php">Already have an account? Login</a>

</form>
</body>
</html>

```

Line-by-Line Key Points

Code / Concept	Why / Explanation
<code>session_start()</code>	Must be first line — before ANY output or HTML
<code>\$errors = []</code>	Empty array — we add error strings with <code>\$errors[]</code>

REQUEST_METHOD === POST	Only process form data when the form was submitted
clean(\$_POST['name'])	Always clean input before using in HTML output
\$_POST['password']	Do NOT clean passwords — cleaning breaks special chars
FILTER_VALIDATE_EMAIL	PHP built-in to check email format (abc@xyz.com)
prepare() + bind_param()	Safe way to use user data in SQL — no injection risk
password_hash()	Converts 'mypass' to '\$2y\$10\$abcd...' (irreversible)
\$stmt->close()	Always close prepared statements to free memory
echo \$success	Safe to echo because we set it ourselves, not from user

● Error Handling Summary

empty(\$name) → 'Full name is required'
 invalid email format → 'Email format is not valid'
 password < 8 chars → 'Password too short'
 passwords don't match → 'Passwords do not match'
 email already in DB → 'Email already registered'
 MySQL insert fails → 'Database error: ' + \$conn->error



login.php / index.php — User Login (READ)



Purpose

Login page checks: does this email+password match a database record?

If yes → save user data in \$_SESSION and redirect to index.php

If no → show 'Invalid credentials' error

This is the R in CRUD (Read — reading the user from the database).



How password_verify() works

The database stores: \$2y\$10\$XKZvqJzSk7...

User types: mypassword123

password_verify('mypassword123', '\$2y\$10\$XKZv...') → true/false

We NEVER decrypt the hash — we just check if they match.



login.php — Full Code with Comments

```

<?php
// =====
// login.php — USER LOGIN
// =====

// Must be the very first line — starts the session system
session_start();

// If user is ALREADY logged in, send them to the dashboard
// isset() checks if a variable exists and is not null
if (isset($_SESSION['user_id'])) {
    header('Location: index.php');
    exit; // IMPORTANT: always exit after header()
}

require_once 'db.php'; // Load $conn and helper functions

// Initialize error variable
$error = '';
$email = ''; // Keep the email value if login fails (re-fill form)

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // Read and clean form values
    $email = clean($_POST['email']);
    $password = $_POST['password']; // Raw password — do NOT clean

    // — VALIDATION —

    if (empty($email) || empty($password)) {
        $error = 'Both email and password are required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = 'Please enter a valid email address.';
    } else {
        // — FIND USER IN DATABASE —

        // Prepared statement: find user by email
        // We fetch ALL columns (*) so we can check the password
        $stmt = $conn->prepare(
            'SELECT * FROM users WHERE email = ? LIMIT 1'
        );
        // 's' = one String parameter

```

```

$stmt->bind_param('s', $email);
$stmt->execute();

// get_result() returns a result object we can fetch from
$result = $stmt->get_result();
// fetch_assoc() returns one row as an associative array
// e.g. ['id'=>1, 'name'=>'Alice', 'email'=>'...', ...]
$user = $result->fetch_assoc();
$stmt->close();

// — CHECK PASSWORD —————

// $user is NULL if no row was found (wrong email)
// password_verify() compares plain password with hash
if ($user && password_verify($password, $user['password'])) {

    // ✓ LOGIN SUCCESS
    // Save user data into the session
    // Session data is stored on the SERVER (not in browser)
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['user_name'] = $user['name'];
    $_SESSION['user_role'] = $user['role'];

    // Redirect to the users list page
    redirect('index.php');

} else {
    // ✗ Wrong email or wrong password
    // We show a VAGUE message on purpose:
    // Never say 'email not found' – it reveals which emails exist
    $error = 'Invalid email or password.';
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Login</title></head>
<body>

<h2>Login</h2>

<!-- Show error message if login failed -->
<?php if ($error): ?>
    <p style="color:red">✗ <?php echo $error; ?></p>
<?php endif; ?>

<form method="post" action="">

    Email: <br>
    <!-- value=... re-fills email so user doesn't retype it -->
    <input type="email" name="email"
        value="<?php echo $email; ?>" required><br><br>

    Password: <br>
    <!-- Never re-fill password fields for security -->
    <input type="password" name="password" required><br><br>

    <input type="submit" value="Login">
    <a href="register.php">Don't have an account? Register</a>

</form>
</body>
</html>

```

Line-by-Line Key Points

Code / Concept	Why / Explanation
<code>if (isset(\$_SESSION['user_id']))</code>	Auto-redirect if already logged in — prevents double login
<code>\$_POST['password']</code> (no clean)	Raw password needed — cleaning can break special chars
<code>LIMIT 1</code> in SQL	We only need one user — faster query
<code>\$result->fetch_assoc()</code>	Returns row as array: <code>\$user['name'], \$user['email']...</code>
<code>\$user</code> && <code>password_verify(...)</code>	Must check <code>\$user</code> is NOT null before verifying password
<code>\$_SESSION['user_id']</code>	Saves user ID on server — proves they are logged in
<code>redirect('index.php')</code>	Our helper function: <code>header()</code> + <code>exit</code>
<code>'Invalid email or password.'</code>	Vague on purpose — don't reveal if email exists or not

Error Handling Summary

Both fields empty → 'Both email and password are required'
Bad email format → 'Please enter a valid email address'
Email not in database → 'Invalid email or password'
Wrong password → 'Invalid email or password' (same vague message)



index.php — Show All Users + Delete (READ + DELETE)



Purpose

Protected page — only accessible when logged in.
Shows all users in an HTML table.
Has Edit and Delete action buttons for each row.
This is R (Read all) + D (Delete) in CRUD.



Full Code with Comments

```
<?php
// =====
// index.php – USERS LIST (READ + DELETE)
// =====

session_start(); // Start session FIRST

// — AUTH GUARD —————
// If user_id is NOT in session, they are not logged in
// Redirect to login page immediately
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}

require_once 'db.php';

// — HANDLE DELETE —————
// Check if a delete request came from a link: index.php?delete=5
// isset() checks the key exists in $_GET
// intval() converts the value to an integer (security: removes SQL tricks)
if (isset($_GET['delete'])) {
    $id = intval($_GET['delete']); // Safe integer

    // Extra safety: never delete if id is 0 or negative
    if ($id > 0) {
        // Prevent deleting your own account
        if ($id === (int)$_SESSION['user_id']) {
            $_SESSION['msg'] = 'You cannot delete your own account.';
        } else {
            $stmt = $conn->prepare('DELETE FROM users WHERE id = ?');
            $stmt->bind_param('i', $id); // 'i' = Integer parameter
            if ($stmt->execute()) {
                $_SESSION['msg'] = 'User deleted successfully.';
            } else {
                $_SESSION['msg'] = 'Delete failed: ' . $conn->error;
            }
            $stmt->close();
        }
    }

    // Redirect back to the same page to avoid re-delete on refresh
    // This pattern is called PRG (Post-Redirect-Get)
    header('Location: index.php');
    exit;
}

// — FETCH ALL USERS —————
// Simple SELECT – no user input so no prepared statement needed here
$result = $conn->query('SELECT * FROM users ORDER BY created_at DESC');

// Check the query ran without error
```

```

if (!$result) {
    die('Query failed: ' . $conn->error);
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Users List</title></head>
<body>

<h2>Users List</h2>

<!-- Show logged-in user's name from session -->
<p>Welcome, <strong><?php echo clean($_SESSION['user_name']); ?></strong>!
    <a href="logout.php">Logout</a></p>

<!-- Show flash message from session (set after delete or update) -->
<?php if (isset($_SESSION['msg'])): ?>
    <p style="color:green">✔ <?php echo $_SESSION['msg']; ?></p>
    <?php unset($_SESSION['msg']); // Delete message after showing it ?>
<?php endif; ?>

<a href="register.php">+ Add New User</a>

<!-- Users table -->
<table border="1" cellpadding="8">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Role</th>
        <th>Registered</th>
        <th>Actions</th>
    </tr>

    <?php
    // Loop through every row returned by the query
    // mysqli_fetch_assoc() returns one row at a time as array
    // Returns NULL when there are no more rows (loop ends)
    while ($row = $result->fetch_assoc()): ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <!-- Always echo through clean() to prevent XSS -->
            <td><?php echo clean($row['name']); ?></td>
            <td><?php echo clean($row['email']); ?></td>
            <td><?php echo clean($row['phone']) ?? '-'; ?></td>
            <td><?php echo $row['role']; ?></td>
            <!-- date() formats a MySQL timestamp into readable date -->
            <td><?php echo date('d M Y', strtotime($row['created_at'])); ?></td>
            <td>
                <!-- Edit: goes to edit.php with the user's id in URL -->
                <a href="edit.php?id=<?php echo $row['id']; ?>">Edit</a>
                &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&
                <!-- Delete: goes back to index.php?delete=ID -->
                <!-- confirm() shows a popup asking the user to confirm -->
                <a href="index.php?delete=<?php echo $row['id']; ?>"
                    onclick="return confirm('Delete this user?')">Delete</a>
            </td>
        </tr>
    </tr>
    <?php endwhile; ?>

</table>

<!-- Show total count -->
<p>Total users: <strong><?php echo $result->num_rows; ?></strong></p>

</body>

```

</html>

Line-by-Line Key Points

Code / Concept	Why / Explanation
Auth guard at top	Every protected page must check session first — redirect if missing
intval(\$_GET['delete'])	Converts URL param to integer — prevents SQL like ?delete=1 OR 1=1
\$id === (int)\$_SESSION[...]	Prevents logged-in user from deleting their own account
'i' in bind_param	'i' means Integer. Use 's' for strings, 'd' for float
header() + exit after delete	PRG pattern — stops page from re-running the delete on F5 refresh
\$_SESSION['msg']	Flash message: set before redirect, show and delete after redirect
unset(\$_SESSION['msg'])	Deletes the message after showing so it only appears once
clean(\$row['name'])	Always sanitize DB values before echoing — DB could have old bad data
\$row['phone'] ?? '-'	Null coalescing: shows '-' if phone is NULL in database



edit.php — Pre-filled Edit Form (READ one record)



Purpose

Receives a user ID from the URL: edit.php?id=3

Fetches that user's data from the database.

Displays a form pre-filled with the current values.

This is R (Read one) in CRUD — the Update happens in update.php.



Full Code with Comments

```
<?php
// =====
// edit.php – EDIT USER FORM (loads current data)
// =====

session_start();

// Auth guard: redirect if not logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}

require_once 'db.php';

// — GET AND VALIDATE THE ID FROM URL —————
// The URL looks like: edit.php?id=5
// $_GET['id'] reads the id value from the URL
// intval() converts it to an integer (removes SQL injection risk)
$id = intval($_GET['id'] ?? 0);

// If id is 0 or missing, it is not a valid request
if ($id <= 0) {
    // Set error message and redirect back to list
    $_SESSION['msg'] = 'Invalid user ID.';
    header('Location: index.php');
    exit;
}

// — FETCH THE USER FROM DATABASE —————
$stmt = $conn->prepare('SELECT * FROM users WHERE id = ? LIMIT 1');
$stmt->bind_param('i', $id); // 'i' = Integer
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc(); // Get the row as array
$stmt->close();

// If no row was returned, the ID does not exist in the database
if (!$user) {
    $_SESSION['msg'] = 'User not found.';
    header('Location: index.php');
    exit;
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Edit User</title></head>
<body>

<h2>Edit User: <?php echo clean($user['name']); ?></h2>
```

```

<!-- Show any error from a previous failed update -->
<?php if (isset($_SESSION['edit_errors'])): ?>
    <div style="color:red">
        <?php foreach ($_SESSION['edit_errors'] as $e): ?>
            <p>✖ <?php echo $e; ?></p>
        <?php endforeach; ?>
    </div>
    <?php unset($_SESSION['edit_errors']); // Clear after showing ?>
<?php endif; ?>

<!-- FORM sends data to update.php -->
<!-- method=post – form data goes in request body (not URL) -->
<form method="post" action="update.php">

    <!-- Hidden field: passes the user's ID to update.php -->
    <!-- We cannot put id in a visible field – user could change it -->
    <!-- NOTE: Hidden fields can still be changed by the user. -->
    <!-- update.php must ALSO validate the id independently. -->
    <input type="hidden" name="id" value="<?php echo $user['id']; ?>">

    Full Name: <br>
    <!-- value=... pre-fills the field with current data from DB -->
    <input type="text" name="name"
        value="<?php echo clean($user['name']); ?>" required><br><br>

    Email Address: <br>
    <input type="email" name="email"
        value="<?php echo clean($user['email']); ?>" required><br><br>

    Phone: <br>
    <input type="text" name="phone"
        value="<?php echo clean($user['phone'] ?? ''); ?>"><br><br>

    Role: <br>
    <select name="role">
        <!-- selected attribute marks the current role as selected -->
        <option value="user"
            <?php echo $user['role']==='user' ? 'selected' : ''; ?>>User</option>
        <option value="admin"
            <?php echo $user['role']==='admin' ? 'selected' : ''; ?>>Admin</option>
    </select><br><br>

    <!-- Password section: leave blank to keep current password -->
    <hr>
    <p><em>Leave password blank to keep the current password.</em></p>

    New Password (optional): <br>
    <input type="password" name="password"><br><br>

    Confirm New Password: <br>
    <input type="password" name="confirm_password"><br><br>

    <input type="submit" value="Save Changes">
    <a href="index.php">Cancel</a>

</form>
</body>
</html>

```

Line-by-Line Key Points

Code / Concept	Why / Explanation
<code>intval(\$_GET['id'] ?? 0)</code>	?? 0 means: use 0 if 'id' key does not exist in \$_GET

\$id <= 0 check	Catches missing, zero, or negative IDs early
LIMIT 1	We expect only one user per ID — LIMIT makes query faster
if (!\$user)	\$user is NULL if no row returned — handle this case
<input type="hidden" name="id"	Passes the ID to update.php invisibly (user still sees page source)
pre-fill with \$user['name']	Shows current value so user only changes what they want
\$user['role']==='user' ? ...	Ternary to add 'selected' attribute to the correct option
Password blank = keep old	Common pattern: only update password if user types a new one
\$_SESSION['edit_errors']	Errors from update.php are passed back via session to show here



update.php — Process Edit Form (UPDATE)



Purpose

Receives form data from edit.php via POST.
Validates all fields.
Updates the database record.
Redirects back to index.php with a success or error message.
This is the U in CRUD (Update).



Why a Separate File?

edit.php handles displaying the form.
update.php handles processing the data.
This keeps each file focused on one job — easier to read and debug.
This pattern is called Separation of Concerns.



Full Code with Comments

```
<?php
// =====
// update.php – PROCESS EDIT FORM (UPDATE record in database)
// This file should NEVER be opened directly in the browser.
// It only works when the edit.php form is submitted.
// =====

session_start();

// Auth guard
if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}

// — BLOCK DIRECT ACCESS —————
// If someone opens update.php directly in browser:
// REQUEST_METHOD will be 'GET' not 'POST'
// We redirect them away immediately
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: index.php');
    exit;
}

require_once 'db.php';

// — READ FORM DATA —————
$id      = intval($_POST['id'] ?? 0); // Get id from hidden field
$name    = clean($_POST['name'] ?? '');
$email   = clean($_POST['email'] ?? '');
$phone   = clean($_POST['phone'] ?? '');
$role    = clean($_POST['role'] ?? 'user');
$password = $_POST['password'] ?? ''; // Raw – do not clean
$confirm = $_POST['confirm_password'] ?? '';

// — VALIDATION —————
$errors = [];

// Check id is valid
if ($id <= 0) {
    $errors[] = 'Invalid user ID.';
}
```

```

// Check name
if (empty($name)) {
    $errors[] = 'Full name is required.';
}

// Check email format
if (empty($email)) {
    $errors[] = 'Email is required.';
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Email format is not valid.';
}

// Check role – only 'user' or 'admin' allowed
if (!in_array($role, ['user', 'admin'])) {
    $role = 'user'; // Reset to safe default if tampered
}

// Password is OPTIONAL in update
// Only validate if user typed something
$change_password = false;
if (!empty($password)) {
    if (strlen($password) < 8) {
        $errors[] = 'New password must be at least 8 characters.';
    } elseif ($password !== $confirm) {
        $errors[] = 'New passwords do not match.';
    } else {
        // Password is valid and will be changed
        $change_password = true;
    }
}

// — CHECK DUPLICATE EMAIL (excluding current user) —
if (empty($errors)) {
    // This query finds another user with the same email
    // 'AND id != ?' excludes the current user from the check
    // Without this, we would always get a duplicate error on save
    $stmt = $conn->prepare(
        'SELECT id FROM users WHERE email = ? AND id != ? LIMIT 1'
    );
    // 'si' = String (email) + Integer (id)
    $stmt->bind_param('si', $email, $id);
    $stmt->execute();
    $stmt->store_result();
    if ($stmt->num_rows > 0) {
        $errors[] = 'This email is already used by another account.';
    }
    $stmt->close();
}

// — IF ERRORS: go back to edit page —
if (!empty($errors)) {
    // Store errors in session so edit.php can display them
    $_SESSION['edit_errors'] = $errors;
    // Redirect back to edit page with the same id
    header('Location: edit.php?id=' . $id);
    exit;
}

// — BUILD AND RUN UPDATE QUERY —
if ($change_password) {
    // User typed a new password – hash it and include in UPDATE
    $hashed = password_hash($password, PASSWORD_DEFAULT);

    $stmt = $conn->prepare(
        'UPDATE users SET name=?, email=?, phone=?, role=?, password=? WHERE id=?'
    );
}

```

```

    // 'sssss' = 5 Strings, then 'i' = 1 Integer (id)
    $stmt->bind_param('ssssi', $name, $email, $phone, $role, $hashed, $id);

} else {
    // No password change – do NOT touch the password column
    $stmt = $conn->prepare(
        'UPDATE users SET name=?, email=?, phone=?, role=? WHERE id=?'
    );
    // 'ssss' = 4 Strings, then 'i' = 1 Integer (id)
    $stmt->bind_param('ssssi', $name, $email, $phone, $role, $id);
}

if ($stmt->execute()) {
    // affected_rows: number of rows changed
    // It is 0 if data was same (no real change) – still a success
    $_SESSION['msg'] = 'User updated successfully.';
} else {
    $_SESSION['msg'] = 'Update failed: ' . $conn->error;
}
$stmt->close();

// Always redirect after a POST (PRG pattern)
// This prevents re-running the UPDATE if user presses F5
header('Location: index.php');
exit;
?>

```

Line-by-Line Key Points

Code / Concept	Why / Explanation
REQUEST_METHOD !== POST	Blocks direct browser access — update.php is POST-only
intval(\$_POST['id'] ?? 0)	?? 0 default + intval() — handles missing or non-numeric id safely
AND id != ? in SELECT	Checks email is unique BUT ignores the current user's own email
\$change_password flag	Boolean flag decides which SQL query to use (with/without password)
Two different prepare() calls	One includes password column, one doesn't — never overwrite with empty
password_hash() again	New password is hashed the same way as registration
\$stmt->execute() check	Always check if execute() succeeded — log/show \$conn->error if not
\$_SESSION['edit_errors']	Pass errors back to edit.php without losing the form data
header() + exit at end	PRG pattern — redirect after any POST action always

Error Handling Summary

Direct browser access	→ Redirect to index.php
Invalid id (<=0)	→ 'Invalid user ID'
Empty name	→ 'Full name is required'
Bad email format	→ 'Email format is not valid'
Email used by another user	→ 'Email already used by another account'
New password < 8 chars	→ 'New password must be at least 8 characters'
Passwords don't match	→ 'New passwords do not match'

MySQL UPDATE fails → 'Update failed: ' + \$conn->error



logout.php — Destroy Session & Redirect



Full Code with Comments

```
<?php
// =====
// logout.php – LOGOUT USER
// =====

// Start the session so we can destroy it
session_start();

// 1. Unset all session variables (clear the data)
//     $_SESSION becomes an empty array after this
session_unset();

// 2. Destroy the session (removes session file from server)
session_destroy();

// 3. Send user back to login page
header('Location: login.php');
exit;
?>
```



session_unset vs session_destroy

session_unset() → clears the data (\$_SESSION becomes [])

session_destroy() → deletes the session file from the server

Use BOTH together for a complete logout.



How All Files Connect — Flow Diagram



Shared Pattern on Every Protected Page

```
// — TOP OF EVERY PROTECTED PAGE (index, edit, update) —
<?php
session_start(); // 1. Start session
if (!isset($_SESSION['user_id'])) { // 2. Auth check
    header('Location: login.php');
    exit;
}
require_once 'db.php'; // 3. Database
// ... rest of the page logic ...
?>
```



Quick Reference Cheat Sheet

PHP Security & Validation

Code	Purpose
<code>password_hash(\$p, PASSWORD_DEFAULT)</code>	Encrypt password before storing
<code>password_verify(\$plain, \$hash)</code>	Check password against stored hash
<code>filter_var(\$e, FILTER_VALIDATE_EMAIL)</code>	Validate email format — returns false if invalid
<code>intval(\$x)</code>	Convert to integer — removes injection tricks
<code>clean(\$x) / htmlspecialchars(\$x)</code>	Prevent XSS — converts < > to < >
<code>\$conn->prepare() + bind_param()</code>	Prevent SQL injection — never concat user input in SQL
<code>isset(\$var)</code>	Check variable exists (and not null)
<code>empty(\$var)</code>	Check variable is empty string, 0, null, or []
<code>\$x ?? \$default</code>	Use \$default if \$x is null or undefined

bind_param Type Characters

Character	Data Type
s	String — VARCHAR, TEXT, ENUM
i	Integer — INT, TINYINT
d	Double — FLOAT, DECIMAL
b	Blob — binary data

Session Cheat Sheet

Code	What it Does
<code>session_start()</code>	Must be at top of EVERY file that uses sessions
<code>\$_SESSION['key'] = 'value'</code>	Save data to session
<code>isset(\$_SESSION['key'])</code>	Check if session value exists
<code>unset(\$_SESSION['key'])</code>	Delete one session variable
<code>session_unset()</code>	Clear ALL session variables
<code>session_destroy()</code>	Delete session file from server (full logout)

MySQLi Prepared Statements — Pattern

```
// 1. Prepare SQL with ? for each value
$stmt = $conn->prepare('SELECT * FROM users WHERE email = ? AND id = ?');

// 2. Bind values to the ? placeholders
//    First arg: 'si' = String + Integer (one char per ? in the SQL)
$stmt->bind_param('si', $email, $id);

// 3. Run the query
$stmt->execute();

// 4a. For SELECT: get results
$result = $stmt->get_result();
$row     = $result->fetch_assoc(); // One row

// 4b. For INSERT/UPDATE/DELETE: check success
if ($stmt->execute()) {
    echo 'Success! Rows affected: ' . $stmt->affected_rows;
} else {
    echo 'Error: ' . $stmt->error;
}

// 5. Always close
$stmt->close();
$conn->close(); // Optional – PHP closes it at end of script
```



Common Errors & How to Fix Them

Error Message / Problem	How to Fix
Headers already sent	session_start() must be line 1 — no echo or HTML before it
Undefined index 'name'	Use isset() or ?? before reading \$_POST/\$_GET values
password_verify() always false	You cleaned() the password before hashing — don't clean passwords
Duplicate entry for 'email'	Email already in DB — add duplicate check with SELECT first
Call to undefined method get_result	Enable mysqli extension or use store_result/fetch methods
Access denied for root	Wrong DB_USER or DB_PASS in db.php
Unknown database crud_app	Run CREATE DATABASE in phpMyAdmin first
Update runs even when no submit	Check REQUEST_METHOD === POST before processing
F5 after form resubmits the form	Add header(redirect) + exit after every POST (PRG pattern)
id from hidden field can be faked	Always re-validate \$id from database in update.php too
No data shown on index.php	Check \$result is not false — echo \$conn->error to debug

✓ You are ready when you can...

1. Register a new user and see the hashed password in phpMyAdmin
2. Login and be redirected to index.php (session is set)
3. See all users listed in the table
4. Click Edit — form is pre-filled with correct data
5. Save changes — database is updated, redirected to list
6. Delete a user — confirmation popup, row is removed
7. Logout — session is gone, can't access index.php anymore